Memory Forensics Investigation Report:
Analysis of In-Memory Attacker Techniques Including C2 Activity, Process Injection, and Privilege Escalation

Prepared by: Jerusalem Madessa

Disclaimer

This investigation was conducted in a controlled and authorized environment for defensive security analysis. All activities were performed on simulated systems to evaluate potential threats and understand attacker behavior.

Executive Summary

This investigation analyzed a system memory image to identify signs of compromise, uncover malicious activity, and reconstruct the attacker's behavior. The objective was to determine whether the system had been compromised, how the attacker operated, and what level of access was achieved.

Analysis of the memory image revealed multiple indicators of compromise, including suspicious processes, unauthorized network communication, privilege escalation, and evidence of process injection. A process was identified establishing communication with an external IP address over port 4444, a port commonly associated with reverse shell activity. This confirmed that the system was actively communicating with an external command-and-control (C2) server.

Further analysis showed a sequence of attacker actions, including system reconnaissance using standard utilities, execution of a short-lived process consistent with privilege escalation techniques, and deployment of a secondary payload with elevated privileges. Memory artifacts also confirmed that malicious code was injected into a legitimate system process, allowing the attacker to maintain stealth and evade detection.

The attacker gained initial access, established external communication, performed privilege reconnaissance, escalated privileges, and deployed a secondary payload while maintaining persistence through process injection.

Scope

This investigation focused on analyzing a Windows memory image to identify malicious activity and reconstruct the attack chain. The scope included:

- Process and memory analysis
- Network connection investigation
- Privilege escalation detection
- Process injection identification
- Suspicious file extraction and analysis

All analysis was performed on authorized systems within a controlled environment.

Objectives

- Identify suspicious and hidden processes
- Detect unauthorized external communication
- Analyze attacker behavior from memory artifacts
- Identify privilege escalation activity
- Detect process injection techniques
- Reconstruct the full attack sequence

Tools Used

- Volatility Framework – Memory analysis
- Paraben E3 – Digital forensic analysis
- FTK Imager – Memory acquisition and inspection
- DensityScout – File structure analysis

Methodology

The investigation started by loading the memory image and reviewing the overall system state. The first goal was to understand what was running on the system at the time of capture and establish a baseline of normal activity. Standard Windows processes such as svchost.exe, lsass.exe, and explorer.exe were identified to separate expected behavior from anything unusual.

Once a baseline was established, the focus shifted to identifying suspicious processes. Process listings were reviewed and compared to detect executables with unusual names, unexpected locations, or abnormal behavior. Special attention was given to processes running from user directories, as these are commonly used to execute malicious files.

To detect stealth techniques, multiple process enumeration methods were compared. Instead of relying on a single view, results from different Volatility plugins were analyzed together. Any mismatch between these views indicated potential hidden or unlinked processes, which is a common evasion technique used by malware.

After identifying suspicious processes, the investigation moved to analyzing how those processes were related. Parent-child relationships were examined to understand execution flow. This helped identify whether processes were launched as part of normal system behavior or as part of a coordinated attack sequence.

Network activity was then analyzed to determine whether the system was communicating externally. Active and historical connections were reviewed to identify unusual destinations or ports. Connections associated with suspicious processes were prioritized, especially those

communicating over uncommon ports such as 4444, which is often linked to reverse shells or remote access tools.

Suspicious executables identified in memory were extracted for further analysis. Their file paths and origins were reviewed to determine how they entered the system. Files originating from user download directories were treated as high-risk and analyzed more closely.

Privilege analysis was performed next to determine whether any process gained elevated access. By reviewing process privileges, it was possible to identify whether the attacker moved from a standard user context to a higher privilege level. This step is critical because privilege escalation allows attackers to gain broader control over the system.

Memory scanning techniques were then used to identify signs of code injection. Evidence of malicious data within legitimate processes, such as svchost.exe, indicated that the attacker was using process injection to remain hidden and bypass detection mechanisms.

Finally, all findings were correlated to reconstruct the full attack sequence. Process activity, network connections, privilege changes, and memory artifacts were analyzed together rather than individually. This made it possible to understand not just what happened, but how the attack progressed from initial execution to full system compromise.

Key Findings

1. Unauthorized External Communication (C2 Activity)

A process was identified communicating with an external IP address over port 4444. This port is commonly associated with reverse shell connections and remote control activity.

This confirms that the system was actively communicating with an external command-and-control server.

Risk Level: High

2. Suspicious Process Execution and Behavior

Multiple processes were identified that did not align with normal system behavior. These processes showed unusual execution paths and were part of a sequence of coordinated activity.

Commands such as whoami.exe were executed, indicating that the attacker was performing reconnaissance to determine privilege levels.

Risk Level: High

3. Privilege Escalation Detected

Analysis revealed that a process initially running with limited privileges was followed by another process executing with elevated privileges. This indicates that the attacker successfully escalated access.

Privilege escalation allows attackers to gain control over critical system components.

Risk Level: High

4. Process Injection into Legitimate System Process

Memory artifacts showed that malicious code was present within the memory space of a legitimate process (svchost.exe).

This indicates process injection, a technique used to hide malicious activity within trusted system processes to evade detection.

Risk Level: High

5. Suspicious File Presence in Memory

Files extracted from memory showed characteristics that required further analysis. While not heavily obfuscated, their presence in memory alongside malicious behavior indicates potential payload activity.

Risk Level: Medium

Remediation Recommendations

- Monitor and restrict outbound connections to unknown external hosts
- Implement endpoint detection and response (EDR) solutions
- Enforce least privilege access controls
- Monitor for abnormal process behavior and parent-child relationships
- Detect and block process injection techniques
- Perform regular memory and threat hunting analysis

Validation & Analysis

- External communication confirmed through memory artifacts
- Process behavior validated through relationship analysis
- Privilege escalation confirmed through privilege comparison
- Injection activity verified through memory inspection
- Findings correlated across multiple analysis techniques

Security Improvements

- Improved visibility into system-level activity
- Detection of advanced attack techniques (injection, escalation)
- Enhanced ability to identify attacker behavior
- Stronger incident detection through memory analysis

Conclusion

This investigation confirmed that the system was compromised and actively controlled by an external attacker. The attacker established remote access, performed reconnaissance, escalated privileges, and used process injection to maintain persistence and evade detection.

The findings highlight how advanced threats can operate entirely within system memory, making them difficult to detect using traditional security tools. Memory forensics provides critical visibility into these attacks and plays a key role in incident response and threat hunting.